Trabajo backend grupo 02

Responsabilidades:

Coordinación y seguimiento: Alejandro Linconguir
Dominio y datos: Byron Salgado
API y lógico de negocio: Vicente Barrientos
Calidad y pruebas: Benjamin Jara
Documentación e integración: Alejandro Linconguir

Hito 1 — Definición del proyecto (completado)

Definir problemática, actores, alcance, entidades, relaciones, reglas de negocio, validaciones y requisitos principales de la API.

Hito 2 — Construcción de la API

Crear la estructura del backend, implementar las entidades, almacenamiento en memoria, repositorios, servicios, routers y primeros endpoints.

Hito 3 — Validaciones y manejo de errores

Implementar las validaciones, reglas de negocio y respuestas HTTP correspondientes a operaciones exitosas y errores.

Hito 4 — API completa

Completar los endpoints requeridos y definir el contrato de requests y responses de la API.

Hito 5 — Consultas avanzadas

Implementar filtros, ordenamiento y paginación en el endpoint de consulta del catálogo.

Hito 6 — Pruebas

Probar las funcionalidades, errores, reglas de negocio y consultas avanzadas mediante una colección de pruebas.

Hito 7 — Integración y entrega

Integrar el trabajo del grupo, completar README y Swagger/OpenAPI, revisar Git y preparar la presentación y demostración.

RetroGames — Sistema de arriendo de videojuegos clásicos

Problemática general
Un local dedicado al arriendo de videojuegos clásicos necesita llevar un registro organizado de sus juegos, clientes y arriendos. Actualmente, la gestión manual puede dificultar conocer qué juegos están disponibles, qué juegos están arrendados, quién tiene cada juego y cuándo corresponde devolverlos.

Objetivo
Desarrollar una API REST que permita al local **administrar su catálogo de videojuegos, clientes, categorías y arriendos**, controlando la disponibilidad de los juegos y evitando situaciones como arriendos duplicados.

Reglas de negocio
1. Un videojuego solo puede ser arrendado si se encuentra disponible.
2. Un cliente no puede tener más de un arriendo activo simultáneamente.
3. La fecha de devolución no puede ser anterior a la fecha de arriendo.
4. Un arriendo debe estar asociado a un cliente existente.
5. Un arriendo debe estar asociado a un videojuego existente.
6. Al devolver un videojuego, su estado vuelve a ser: Disponible.

Validaciones
La API contempla validaciones de:
- **Longitud:** títulos y otros campos de texto.
- **Valores permitidos:** plataformas y estados.
- **Valores numéricos:** IDs y parámetros de paginación.
- **Formato:** correo electrónico y datos definidos mediante Pydantic.
- **Existencia de relaciones:** categoría, cliente y videojuego.
- **Reglas de negocio:** disponibilidad, arriendos activos y fechas.

P1 — ¿Qué situación concreta ocurre?
El local de videojuegos clásicos administra sus juegos y arriendos de manera manual, lo que dificulta conocer rápidamente qué videojuegos están disponibles, cuáles están actualmente arrendados, quién los tiene y cuándo deben ser devueltos.

P2 — ¿Quiénes tienen el problema?
Tendremos dos actores principales:

Encargado del local
Se encarga de:
- Registrar videojuegos.
- Modificar información.
- Registrar clientes.
- Registrar arriendos.
- Registrar devoluciones.
- Consultar disponibilidad.

Cliente
Puede:
- Consultar el catálogo de videojuegos.
- Solicitar un arriendo.
- Consultar información asociada a sus arriendos.
- Realizar la devolución de un videojuego.

P3 — Consecuencias
Podemos establecer:
1. El encargado puede tener dificultades para saber qué juegos están disponibles.
2. Se pueden producir errores al registrar arriendos.
3. Puede ser difícil identificar qué cliente tiene un juego determinado.
4. Se pueden generar problemas con devoluciones atrasadas.
5. Puede resultar difícil mantener actualizado el catálogo.

P4 — ¿Qué información maneja?
Cliente
Videojuego
Arriendo
Categoría

Cliente:
- Identificador del cliente.
- Nombre.
- Apellido.
- Correo.
- Teléfono.
- Estado.

Videojuego:
- Identificador del videojuego.
- Título.
- Plataforma.
- Año de lanzamiento.
- Estado/disponibilidad.
- Categoría.

Categoría:
- Identificador de categoría.
- Nombre.
- Descripción.
- Edad mínima.
- Estado.

Arriendo:
- Identificador del arriendo.
- Cliente que realiza el arriendo.
- Videojuego arrendado.

- Fecha de arriendo.
- Fecha de devolución.
- Estado del arriendo.

P5 — ¿Qué debe permitir la API?

La guía pide **al menos 8 requisitos** escritos como "El sistema debe permitir...".
Podemos dejar:

1. El sistema debe permitir registrar videojuegos.
2. El sistema debe permitir consultar videojuegos.
3. El sistema debe permitir modificar videojuegos.
4. El sistema debe permitir eliminar videojuegos.
5. El sistema debe permitir registrar clientes.
6. El sistema debe permitir registrar categorías.
7. El sistema debe permitir registrar arriendos.
8. El sistema debe permitir registrar devoluciones.
9. El sistema debe permitir consultar los arriendos activos.
10. El sistema debe permitir consultar la disponibilidad de los videojuegos.
11. El sistema debe permitir consultar los arriendos realizados por un cliente.
12. El sistema debe permitir filtrar y ordenar el catálogo de videojuegos.

Con eso tenemos bastante material para sacar los endpoints.

P6 — ¿Qué NO hará el sistema?

Esto también es importante porque evita que el proyecto se vuelva gigante.
Podríamos definir:

La primera versión del sistema no contempla:

1. Pagos en línea.
2. Aplicación móvil o interfaz gráfica para clientes.
3. Sistema de autenticación y cuentas de usuario.
4. Gestión de envíos a domicilio.
5. Base de datos externa.
6. **Nota sobre el Alcance:** Aunque en las etapas iniciales del proyecto se contempló almacenamiento en memoria, la versión final evoluciona a una persistencia relacional completa con PostgreSQL, gestionada a través de contenedores en Docker, mapeo de datos con SQLAlchemy y control de versiones del esquema mediante Alembic.

La última es especialmente importante porque la guía dice explícitamente que para esta entrega deben utilizar **almacenamiento en memoria mediante listas o diccionarios** y no exige base de datos.

P7 — ¿Cómo se sabrá que la solución funciona?

La solución se considerará funcional cuando se cumplan los siguientes criterios:

1. El sistema debe permitir crear, consultar, modificar y eliminar videojuegos correctamente.
2. El sistema debe permitir registrar clientes y evitar correos duplicados.
3. El sistema debe impedir arrendar un videojuego que no esté disponible.
4. El sistema debe impedir que un cliente tenga más de un arriendo activo.
5. El sistema debe rechazar una fecha de devolución anterior a la fecha de arriendo.

6. El sistema debe validar que el cliente y el videojuego asociados a un arriendo existan.
7. El sistema debe permitir consultar el catálogo utilizando filtros, ordenamiento y paginación.
8. Las operaciones inválidas deben entregar códigos HTTP y mensajes de error controlados.
9.

Arquitectura del sistema (4 capas)

El proyecto está estructurado siguiendo los principios de separación de responsabilidades en cuatro capas. Esta organización permite mantener el código ordenado, facilitar su mantenimiento y separar las responsabilidades de cada componente.

- **Presentación (`app/routers/` y `app/schemas/`):** Contiene los endpoints desarrollados con FastAPI y los esquemas Pydantic (DTOs), encargados de validar y estructurar los datos de entrada y salida. También permite exponer la documentación de la API mediante OpenAPI/Swagger.
- **Aplicación (`app/services/`):** Contiene los servicios que implementan los casos de uso y coordinan las operaciones del sistema, aplicando las reglas necesarias para cada funcionalidad.
- **Dominio (`app/domain/`):** Contiene las entidades y las reglas fundamentales del negocio, representando los elementos principales del sistema de arriendo de videojuegos.
- **Infraestructura (`app/repositories/` y componentes de persistencia):** Se encarga del acceso y almacenamiento de los datos, la comunicación con la base de datos PostgreSQL y la implementación de los repositorios. Para ello se utilizan SQLAlchemy y las herramientas de migración correspondientes.
-

Esta separación permite que los routers no tengan que gestionar directamente las consultas a la base de datos y que cada capa tenga una responsabilidad definida.

Guía de instalación y ejecución local

Para instalar y ejecutar el proyecto de forma local desde Git Bash, se deben seguir los siguientes pasos:

1. Clonar el repositorio

```shell
git clone
https://github.com/alejandrolinconguir/Proyecto_Backend.git
cd Proyecto_Backend
```

2. Configurar las variables de entorno

Copiar el archivo de ejemplo para crear el archivo de configuración local:

```shell
cp .env.example .env
```

Completar las variables necesarias de acuerdo con la configuración del proyecto.

3. Activar el entorno virtual e instalar las dependencias

```Shell
source venv/Scripts/activate
pip install -r requirements.txt
```

4. Iniciar la base de datos PostgreSQL mediante Docker

```Shell
docker compose up -d
```

5. Ejecutar las migraciones de la base de datos

```Shell
alembic upgrade head
```

Este comando aplica las migraciones pendientes y actualiza la estructura de la base de datos.

6. Cargar los datos iniciales

Ejecutar el proceso de inicialización de datos (seeders), de acuerdo con la ubicación del archivo correspondiente en el proyecto.

7. Iniciar el servidor de la API

```Shell
uvicorn app.main:app --reload
```

Una vez iniciado el servidor, la API estará disponible localmente.

Documentación interactiva de la API

FastAPI proporciona documentación interactiva para consultar y probar los endpoints disponibles.

- **Swagger UI:** http://127.0.0.1:8000/docs
- **ReDoc:** http://127.0.0.1:8000/redoc